DATA HACKATHON MASTER PLAYBOOK

Authors

Md. Masum Akondha

Tanjil Shaiket

Eastern University

Table of Contents

1. Import Section 3

2. Dataset Loading + Basic Insights 3

3. Target, ID, and Column Extraction 3

4. EDA Phase - Basic, Missing, Unique 3

5. Categorical Column Value Counts 3

6. EDA Plots + Safe Correlation 4

7. Manual Imputation Section 4

8. Data Cleaning + Outlier Handling 4

9. Feature Engineering Demo - Generic Pattern 5

10. Ordinal Encoding Priority Example 5

11. Step 5 - Column Grouping + Preprocessing Pipeline 6

12. Train/Test Split + Baseline Model Comparison 7

13. GridSearchCV + Evaluation 7

14. Submission + Final Checklist 8

15. Generic Regression 9

16. Generic Clustering 11

1. Import Section

```
# ============================================================
# import section
# ============================================================
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder, OrdinalEnco
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.linear_model import LogisticRegression, LinearRegression, SGDRe
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.cluster import KMeans
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report,
    mean_squared_error, mean_absolute_error, r2_score,
    silhouette_score
)
import warnings
warnings.filterwarnings("ignore")
print("Libraries loaded successfully.")
```

2. Dataset Loading + Basic Insights

```
# ============================================================
# dataset loading
# ============================================================
train = pd.read_csv("/content/train.csv")
# test = pd.read_csv("/content/test.csv")  # use this if a separate test fil

train.sample(5)
train.shape
train.info()
train.isnull().sum()
train.duplicated().sum()
```

3. Target, ID, and Column Extraction

Use this pattern after setting the target and optional ID columns. The examples use generic placeholders only.

```
# ============================================================
# target and id setup
# ============================================================
TARGET = "Target"          # replace with the actual target column name duri
id_cols = ["ID_Column"]    # use [] if there is no ID column

feature_cols = [c for c in train.columns if c != TARGET and c not in id_cols

num_cols = train[feature_cols].select_dtypes(include=["int64", "float64"]).c
cat_cols = train[feature_cols].select_dtypes(include=["object", "category",

print("Numeric columns    :", num_cols)
print("Categorical columns:", cat_cols)
```

4. EDA Phase - Basic, Missing, Unique

```
# Basic descriptive statistics
train.describe()
train.describe(include="object")

# Missing value overview
missing = pd.DataFrame({
    "Data Type": train.dtypes,
    "Missing Values": train.isnull().sum(),
    "Missing (%)": (train.isnull().mean() * 100).round(2)
})
missing = missing[missing["Missing Values"] > 0]
missing.sort_values("Missing (%)", ascending=False)

# Unique value count
unique = pd.DataFrame({
    "Data Type": train.dtypes,
    "Unique Values": train.nunique()
})
unique.sort_values("Unique Values")
```

5. Categorical Column Value Counts

This simple loop shows each categorical column and the count of each category. This replaces the longer category-summary loop.

```
# Categorical column value counts
for col in cat_cols:
    print(f"\n--- {col} ---")
    print(train[col].value_counts())

# Optional: if missing values should also appear as a count, use:
# print(train[col].value_counts(dropna=False))
```

6. EDA Plots + Safe Correlation

```
# Target variable analysis
print(train[TARGET].value_counts())
print((train[TARGET].value_counts(normalize=True) * 100).round(2))

sns.countplot(data=train, x=TARGET)
plt.title("Target Class Balance")
plt.show()

# Numerical feature analysis
for col in num_cols:
    sns.histplot(data=train, x=col, kde=True)
    plt.title(col)
    plt.show()

# Numerical feature vs Target
for col in num_cols:
    sns.boxplot(data=train, x=TARGET, y=col)
    plt.title(col)
    plt.show()

# Categorical feature vs Target
for col in cat_cols:
    plt.figure(figsize=(10, 4))
    sns.countplot(data=train, x=col, hue=TARGET)
    plt.title(col)
    plt.xticks(rotation=45)
    plt.show()

# ============================================================
# Correlation analysis - safe version
# ============================================================
```

```
corr_cols = list(num_cols)
# Add Target only if Target is numeric. If Target is text, keep only numeric
if TARGET in train.columns and pd.api.types.is_numeric_dtype(train[TARGET]):
    if TARGET not in corr_cols:
        corr_cols.append(TARGET)

corr_matrix = train[corr_cols].corr(numeric_only=True)
display(corr_matrix)

plt.figure(figsize=(10, 6))
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```

7. Manual Imputation Section

This section is for manual checking or creating extra imputed columns. The same statistic from X_train should also be used for X_test.

```
# ============================================================
# Manual mean/median imputation example
# ============================================================
# Calculate statistics from X_train only
mean_feature_1 = X_train["Feature_1"].mean()
median_feature_2 = X_train["Feature_2"].median()

# Fill missing values in X_train
X_train["Feature_1_mean"] = X_train["Feature_1"].fillna(mean_feature_1)
X_train["Feature_2_median"] = X_train["Feature_2"].fillna(median_feature_2)

# Fill missing values in X_test using the same X_train statistics
X_test["Feature_1_mean"] = X_test["Feature_1"].fillna(mean_feature_1)
X_test["Feature_2_median"] = X_test["Feature_2"].fillna(median_feature_2)
```

8. Data Cleaning + Outlier Handling

```
# Duplicate drop
duplicate_count = train.duplicated().sum()
if duplicate_count > 0:
    train = train.drop_duplicates().reset_index(drop=True)
print(f"{duplicate_count} duplicate rows dropped.")

# Optional: replace special missing symbols if needed
# train.replace("Missing_Token", np.nan, inplace=True)
```

```
# Optional: fix text label formatting if needed
# train[TARGET] = train[TARGET].str.replace("Old_Text", "New_Text", regex=Fa

# Outlier detection before handling
for col in num_cols:
    plt.figure(figsize=(6, 2))
    sns.boxplot(x=train[col])
    plt.title(f"Before Handling : {col}")
    plt.show()

# Detect outliers using IQR
for col in num_cols:
    Q1 = train[col].quantile(0.25)
    Q3 = train[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - (1.5 * IQR)
    upper = Q3 + (1.5 * IQR)
    outliers = train[(train[col] < lower) | (train[col] > upper)]
    percentage = (len(outliers) / len(train)) * 100
    print(f"{col}")
    print(f"Outliers   : {len(outliers)}")
    print(f"Percentage : {percentage:.2f}%")
    print("-" * 40)

# Handle outliers using IQR clipping
for col in num_cols:
    Q1 = train[col].quantile(0.25)
    Q3 = train[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - (1.5 * IQR)
    upper = Q3 + (1.5 * IQR)
    if train[col].min() >= 0:
        lower = max(lower, 0)
    train[col] = train[col].clip(lower=lower, upper=upper)

print("Outliers handled using IQR Clipping.")
```

9. Feature Engineering Demo - Generic Pattern

Use placeholder names first, then replace placeholder names with actual column names during the event. These examples are dataset-independent.

```
# Ratio feature: Variable_X compared to Variable_Y
train["Variable_X_to_Variable_Y_Ratio"] = train["Variable_X"] / (train["Vari
```

```
# Per-count feature: Numeric_Feature divided by Count_Feature
train["Numeric_Feature_Per_Count"] = train["Numeric_Feature"] / (train["Coun

# Binary flag feature
train["Has_Feature_1"] = train["Feature_1"].apply(lambda x: 1 if x > 0 else

# Ordered category feature using pd.cut
# Numeric_Feature should be a numeric column.
# bins must be in increasing order.
train["Numeric_Feature_Category"] = pd.cut(
    train["Numeric_Feature"],
    bins=[0, 25, 50, 75, np.inf],
    labels=["Category_A", "Category_B", "Category_C", "Category_D"],
    include_lowest=True
)

# Meaning:
# 0 to 25      -> Category_A
# above 25-50  -> Category_B
# above 50-75  -> Category_C
# above 75     -> Category_D
```

10. Ordinal Encoding Priority Example

Use OrdinalEncoder only when the category has a real order. The order in ordinal_categories controls the encoded numeric values.

```
# ============================================================
# Ordinal Encoding Priority Example
# ============================================================
# ordinal_cols and ordinal_categories must match position-wise.
# 1st column in ordinal_cols uses 1st list in ordinal_categories.
# 2nd column in ordinal_cols uses 2nd list in ordinal_categories.
# 3rd column in ordinal_cols uses 3rd list in ordinal_categories.

ordinal_cols = ["Feature_1_Priority", "Feature_2_Priority", "Feature_3_Prior

ordinal_categories = [
    ["Option_1", "Option_2", "Option_3"],       # Feature_1_Priority: Option
    ["Class_A", "Class_B", "Class_C"],          # Feature_2_Priority: Class_
    ["Category_A", "Category_B", "Category_C"]  # Feature_3_Priority: Catego
]

# Priority rule:
```

```
# First item  -> encoded as 0
# Second item -> encoded as 1
# Third item  -> encoded as 2
# Put the lowest priority category first.
# Put the highest priority category last.

ordinal_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OrdinalEncoder(categories=ordinal_categories))
])

# If a categorical column has no natural order, use OneHotEncoder instead of
```

11. Step 5 - Column Grouping + Preprocessing Pipeline

This follows the initial-handbook style. Fill the column lists manually after checking the dataset. Comments show which list connects to which transformer.

```
# =============================================================
# Step 5: Column Division
# =============================================================
num_mean_cols = []        # normal numeric columns -> mean impute + Standard
num_median_cols = []      # skewed/outlier-heavy numeric -> median impute +
num_scale_only_cols = []  # numeric columns with no missing -> scaling only
binary_cols = []          # 0/1 columns -> most frequent impute
cat_mode_cols = []        # nominal categorical -> mode impute + OneHotEncod
cat_unknown_cols = []     # missing category means Unknown -> constant imput
cat_no_missing_cols = []  # categorical with no missing -> OneHotEncoder onl
ordinal_cols = []         # ordered categorical columns -> OrdinalEncoder
ordinal_categories = []   # Example: [["Option_1", "Option_2", "Option_3"]]

# =============================================================
# Step 5: Transformers
# =============================================================
mean_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="mean")),
    ("scaler", StandardScaler())
])

median_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
```

```
])

scale_transformer = Pipeline([
    ("scaler", StandardScaler())
])

binary_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent"))
])

mode_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

unknown_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="constant", fill_value="Unknown")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

nomissing_transformer = Pipeline([
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

if ordinal_categories:
    ordinal_transformer = Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("encoder", OrdinalEncoder(categories=ordinal_categories))
    ])
else:
    ordinal_transformer = Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("encoder", OrdinalEncoder())
    ])

# ============================================================
# Step 5: ColumnTransformer
# ============================================================
preprocessor = ColumnTransformer([
    ("mean", mean_transformer, num_mean_cols),          # num_mean_cols -> m
    ("median", median_transformer, num_median_cols),    # num_median_cols ->
    ("scale", scale_transformer, num_scale_only_cols),  # num_scale_only_col
    ("binary", binary_transformer, binary_cols),        # binary_cols -> bin
    ("mode", mode_transformer, cat_mode_cols),          # cat_mode_cols -> m
    ("unknown", unknown_transformer, cat_unknown_cols), # cat_unknown_cols -
```

```
    ("nomiss", nomissing_transformer, cat_no_missing_cols), # cat_no_missing
    ("ordinal", ordinal_transformer, ordinal_cols),     # ordinal_cols -> or
], remainder="drop")

print("Preprocessor pipeline is ready.")
print("Check that every needed column is included in one group.")
```

12. Train/Test Split + Baseline Model Comparison

```
X = train.drop(columns=[TARGET])
y = train[TARGET]

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    stratify=y,
    random_state=42
)

if len(id_cols) > 0 and id_cols[0] in X_train.columns:
    test_ids = X_test[id_cols[0]].copy()
    X_train = X_train.drop(columns=id_cols)
    X_test = X_test.drop(columns=id_cols)
else:
    test_ids = None

X_train.shape, X_test.shape

models = {
    "Logistic Regression": LogisticRegression(class_weight="balanced", rando
    "Decision Tree": DecisionTreeClassifier(class_weight="balanced", random_
    "Random Forest": RandomForestClassifier(class_weight="balanced", random_
    "SVM": SVC(class_weight="balanced", random_state=42),
    "KNN": KNeighborsClassifier()
}

results = []
for name, model in models.items():
    pipeline = Pipeline([
        ("preprocessor", preprocessor),
        ("model", model)
    ])
    pipeline.fit(X_train, y_train)
    y_pred = pipeline.predict(X_test)
```

```
    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(y_test, y_pred, average="weighted", zer
        "Recall": recall_score(y_test, y_pred, average="weighted", zero_divi
        "F1 Score": f1_score(y_test, y_pred, average="weighted", zero_divisi
    })

results_df = pd.DataFrame(results).sort_values("F1 Score", ascending=False)
results_df
```

13. GridSearchCV + Evaluation

```
# Classification Hyperparameter Grid
# Each parameter name must match the estimator inside the Pipeline.
logistic_param_grid = {
    "model__C": [0.01, 0.1, 1, 10],
    "model__penalty": ["l2"],
    "model__solver": ["lbfgs"]
}

decision_tree_param_grid = {
    "model__criterion": ["gini", "entropy"],
    "model__max_depth": [None, 5, 10, 20],
    "model__min_samples_split": [2, 5, 10],
    "model__min_samples_leaf": [1, 2, 4]
}

random_forest_param_grid = {
    "model__n_estimators": [100, 200, 300],
    "model__max_depth": [None, 5, 10, 20],
    "model__min_samples_split": [2, 5],
    "model__min_samples_leaf": [1, 2]
}

svm_param_grid = {
    "model__C": [0.1, 1, 10],
    "model__kernel": ["linear", "rbf"],
    "model__gamma": ["scale", "auto"]
}

knn_param_grid = {
    "model__n_neighbors": [3, 5, 7, 9, 11],
    "model__weights": ["uniform", "distance"],
```

```
    "model__metric": ["euclidean", "manhattan"]
}

param_grids = {
    "Logistic Regression": logistic_param_grid,
    "Decision Tree": decision_tree_param_grid,
    "Random Forest": random_forest_param_grid,
    "SVM": svm_param_grid,
    "KNN": knn_param_grid
}

selected_model = "Random Forest"
best_model = models[selected_model]
param_grid = param_grids[selected_model]

pipeline = Pipeline([
    ("preprocessor", preprocessor),
    ("model", best_model)
])

grid_search = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    cv=5,
    scoring="f1_weighted",
    n_jobs=-1
)

grid_search.fit(X_train, y_train)
print("Best Parameters:", grid_search.best_params_)
print("Best CV F1 Score:", grid_search.best_score_)
```

13. GridSearchCV + Evaluation (continued)

```
# Evaluation - safe print style
y_pred = grid_search.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average="weighted", zero_divisio
recall = recall_score(y_test, y_pred, average="weighted", zero_division=0)
f1 = f1_score(y_test, y_pred, average="weighted", zero_division=0)

print(f"Accuracy : {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall   : {recall:.4f}")
print(f"F1 Score : {f1:.4f}")
print()
print(classification_report(y_test, y_pred, zero_division=0))

plt.figure(figsize=(6, 5))
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt="d", cmap="Blu
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

14. Submission + Final Checklist

```
prediction = grid_search.predict(X_test)

if test_ids is not None:
    submission = pd.DataFrame({
        "ID_Column": test_ids,
        TARGET: prediction
    })
else:
    submission = pd.DataFrame({
        TARGET: prediction
    })

submission.to_csv("submission.csv", index=False)
print("Submission file created.")
submission.head()
```

- Simple categorical value_counts loop is used.

- Manual imputation examples use only Feature_1 and Feature_2 placeholders.

- Feature engineering examples use only Variable_X, Variable_Y, Count_Feature, Numeric_Feature, and generic category labels.

- Ordinal examples use only Option_1, Option_2, Option_3, Class_A, Class_B, Class_C, Category_A, Category_B, Category_C.

- Correlation heatmap uses corr_cols = list(num_cols), not num_cols + [TARGET] directly.

- Step 5 manual pipeline follows the initial-handbook style with easy English comments.

15. Generic Regression

Use this section when the Target is a continuous number. This follows the Initial Handbook style: baseline regression models, selected model tuning, evaluation, and prediction. Keep the column names generic until the event dataset is known.

Why this is used: regression predicts a numeric Target. When to use it: use this branch when the problem statement asks for a continuous value. How it works: the preprocessor cleans features first, then the model learns numeric patterns. Output meaning: lower RMSE/MAE is better, and higher R2 Score is better.

Generic Regression - Imports if needed

```
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LinearRegression, SGDRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_scor
```

Generic Regression - Train/Test Split

```
TARGET = "Target"          # continuous numeric target column
id_cols = ["ID_Column"]    # use [] if there is no ID column

X = train.drop(columns=[TARGET])
y = train[TARGET]

# ID columns are usually not model features
X = X.drop(columns=[c for c in id_cols if c in X.columns])

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

X_train.shape, X_test.shape
```

Generic Regression - Preprocessing Pipeline

```
num_cols_reg = X_train.select_dtypes(include=["int64", "float64"]).columns
cat_cols_reg = X_train.select_dtypes(include=["object", "category", "bool"])

num_reg_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

cat_reg_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

reg_preprocessor = ColumnTransformer([
    ("num", num_reg_transformer, num_cols_reg),
    ("cat", cat_reg_transformer, cat_cols_reg)
], remainder="drop")

print("Regression preprocessor is ready.")
```

Generic Regression - Baseline Models

```
models = {
    "Linear Regression": LinearRegression(),
    "SGD Regressor": SGDRegressor(random_state=42),
    "Decision Tree": DecisionTreeRegressor(random_state=42),
    "Random Forest": RandomForestRegressor(random_state=42)
}

results = []
for name, model in models.items():
    pipeline = Pipeline([
        ("preprocessor", reg_preprocessor),
        ("model", model)
    ])
    pipeline.fit(X_train, y_train)
    y_pred = pipeline.predict(X_test)
    results.append({
        "Model": name,
        "RMSE": np.sqrt(mean_squared_error(y_test, y_pred)),
        "MAE": mean_absolute_error(y_test, y_pred),
        "R2 Score": r2_score(y_test, y_pred)
    })
```

```
results_df = pd.DataFrame(results).sort_values("RMSE")
results_df
```

Generic Regression - Hyperparameter Grid

```
linear_regression_param_grid = {}

sgd_regressor_param_grid = {
    "model__alpha": [0.0001, 0.001, 0.01],
    "model__max_iter": [1000, 3000, 5000],
    "model__penalty": ["l2", "l1"]
}

decision_tree_param_grid = {
    "model__max_depth": [None, 5, 10, 20],
    "model__min_samples_split": [2, 5, 10],
    "model__min_samples_leaf": [1, 2, 4]
}

random_forest_param_grid = {
    "model__n_estimators": [100, 200, 300],
    "model__max_depth": [None, 5, 10, 20],
    "model__min_samples_split": [2, 5],
    "model__min_samples_leaf": [1, 2]
}

param_grids = {
    "Linear Regression": linear_regression_param_grid,
    "SGD Regressor": sgd_regressor_param_grid,
    "Decision Tree": decision_tree_param_grid,
    "Random Forest": random_forest_param_grid
}

selected_model = "Random Forest"  # change this based on baseline result
best_model = models[selected_model]
param_grid = param_grids[selected_model]

pipeline = Pipeline([
    ("preprocessor", reg_preprocessor),
    ("model", best_model)
])

grid_search = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
```

```
    cv=5,
    scoring="neg_root_mean_squared_error",
    n_jobs=-1
)

grid_search.fit(X_train, y_train)
print("Best Parameters:", grid_search.best_params_)
print("Best CV Score (neg RMSE):", grid_search.best_score_)
```

Generic Regression - Evaluation

Why: evaluation shows how close predictions are to actual numeric values. RMSE and MAE are error values, so lower is better. R2 Score shows how much variation the model explains, so higher is better.

```
y_pred = grid_search.predict(X_test)

rmse = np.sqrt(mean_squared_error(y_test, y_pred))
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"RMSE    : {rmse:.4f}")
print(f"MAE     : {mae:.4f}")
print(f"R2 Score: {r2:.4f}")

pd.DataFrame({"Actual": y_test, "Prediction": y_pred}).head(10)
```

Generic Regression - Prediction File Pattern

```
# Use this only if a separate test file is given.
# Make sure the same cleaning and feature engineering steps are applied to t

# test_pred = grid_search.predict(test)
# submission = pd.DataFrame({
#     "ID_Column": test_ids["ID_Column"],
#     "Target": test_pred
# })
# submission.to_csv("submission.csv", index=False)
# print("Submission file created.")
# submission.head()
```

16. Generic Clustering

Use this section when there is no Target column and the task is grouping similar rows. This follows the Initial Handbook style: preprocessing, elbow method, final KMeans model, cluster profiling, visualization, and submission pattern.

Why this is used: clustering finds groups without a Target. When to use it: use this branch when the problem asks for segmentation or grouping. How it works: numeric features are scaled first, then KMeans assigns each row to a cluster. Output meaning: Cluster is the group label, and Silhouette Score helps check cluster separation.

Generic Clustering - Imports if needed

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
```

Generic Clustering - Preprocessing + Elbow Method

```
# ============================================================
# Generic Clustering - Preprocessing + Elbow Method
# ============================================================
# Use the existing preprocessor from Step 5.
# If there is no Target column, transform the full train data.
X_cluster = preprocessor.fit_transform(train)

inertia = []
sil_scores = []
for k in range(2, 11):
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    model.fit(X_cluster)
    inertia.append(model.inertia_)
    sil_scores.append(silhouette_score(X_cluster, model.labels_))

plt.figure(figsize=(8, 5))
plt.plot(range(2, 11), inertia, marker="o")
plt.xlabel("Number of Clusters")
plt.ylabel("Inertia")
plt.title("Elbow Method")
plt.grid(True)
plt.show()

plt.figure(figsize=(8, 5))
```

```
plt.plot(range(2, 11), sil_scores, marker="o", color="green")
plt.xlabel("Number of Clusters")
plt.ylabel("Silhouette Score")
plt.title("Silhouette Score vs k")
plt.grid(True)
plt.show()
```

Generic Clustering - Final KMeans Model

```
# Choose best_k after checking Elbow Method and Silhouette Score plot.
best_k = 4
kmeans = KMeans(n_clusters=best_k, random_state=42, n_init=10)
kmeans.fit(X_cluster)

train["Cluster"] = kmeans.labels_
score = silhouette_score(X_cluster, train["Cluster"])
print("Silhouette Score:", round(score, 4))

train["Cluster"].value_counts().sort_index()
```

Generic Clustering - Cluster Profiling

Why: this shows the average numeric pattern inside each cluster. Output meaning: each row is one cluster, and each value is the average of a numeric feature.

```
cluster_summary = train.groupby("Cluster").mean(numeric_only=True)
cluster_summary
```

Generic Clustering - Cluster Visualization

```
# Use two numeric columns for a simple scatter plot.
# Replace Feature_1 and Feature_2 with actual numeric column names during th
plt.figure(figsize=(8, 6))
sns.scatterplot(
    data=train,
    x="Feature_1",
    y="Feature_2",
    hue="Cluster",
    palette="Set2"
)
plt.title("Cluster Visualization")
plt.show()
```

Generic Clustering - Test Cluster Prediction + Submission

```
# Use this only if a separate test file is given.
# The same preprocessor fitted on train is used to transform test.

# test_transformed = preprocessor.transform(test)
# test_cluster = kmeans.predict(test_transformed)

# submission = test.copy()
# submission["Cluster"] = test_cluster
# submission.to_csv("submission.csv", index=False)
# print("Submission file created.")
# submission.head()
```